

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
KHOA KHOA HỌC MÁY TÍNH**



**BÁO CÁO ĐỒ ÁN CUỐI KỲ
MÔN HỌC: CS338.Q22 - NHẬN DẠNG**

**HỆ THỐNG CHATBOT AGENTIC TRONG QUẢN LÝ
THƯƠNG MẠI DỰA TRÊN SPIKEGPT**

Giảng viên hướng dẫn: Dương Việt Hằng, Trần Doãn Thuyên
Nhóm thực hiện: Nhóm 5
Bài báo tham chiếu: SpikeGPT: A Generative Pre-trained Language Model with Spiking Neural Networks
Thời gian thực hiện: 05/2026

STT	Họ và tên	MSSV
1	Phạm Quốc Nam	23520984
2	Nguyễn Khang Hy	23520662

TP. Hồ Chí Minh, tháng 05 năm 2026

Mục lục

TÓM TẮT	1
1 TỔNG QUAN	2
1.1 Bối cảnh và lý do chọn đề tài	2
1.2 Mục tiêu đồ án	2
1.3 Phạm vi và đóng góp	3
1.4 Cấu trúc báo cáo	3
2 CÁC CÔNG TRÌNH LIÊN QUAN	4
2.1 Transformer và mô hình ngôn ngữ tự hồi quy	4
2.2 RWKV như một hướng thay thế attention	4
2.3 SpikeGPT và mô hình ngôn ngữ dựa trên SNN	5
2.4 Tool calling và agentic chatbot	5
2.5 Vị trí của đồ án so với các công trình trước	5
3 CƠ SỞ LÝ THUYẾT	6
3.1 Rào cản của Transformer đối với SNN	6
3.2 Spiking Neural Networks và LIF neuron	7
3.3 Kiến trúc SpikeGPT	8
3.3.1 Binary Embedding	8
3.3.2 Spiking RWKV	8
3.3.3 Spiking Receptance Feed-Forward Network	9
3.4 Ưu điểm và giới hạn kỳ vọng	9
4 BÀI TOÁN VÀ DỮ LIỆU	10
4.1 Bài toán chatbot agentic trong thương mại	10
4.2 Bộ công cụ	11
4.3 Xây dựng dữ liệu	11
4.4 Tiền xử lý và kiểm tra chất lượng	12
4.5 Định dạng ID và phân phối dữ liệu	13

5	PHƯƠNG PHÁP THỰC NGHIỆM	14
5.1	Các mô hình được đánh giá	14
5.2	Nguồn cài đặt và các chỉnh sửa chính	14
5.3	Thiết lập huấn luyện	15
5.4	HeadQK	16
5.5	Hàm mất mát	16
5.6	Các chỉ số đánh giá	18
5.7	Thiết kế ablation	19
5.8	Pipeline suy luận và đánh giá	19
6	KẾT QUẢ VÀ PHÂN TÍCH	21
6.1	Kết quả tổng quan	21
6.2	Phân tích theo từng nhóm metric	22
6.3	Tác động của HeadQK	22
6.4	Ablation về hàm mất mát	23
6.5	Ablation về định dạng ID	24
6.6	Phân tích vì sao SpikeGPT yếu ở Args Exact Match	24
6.7	Các dạng lỗi thường gặp	25
6.8	Đánh giá định tính	25
6.9	So sánh với bài báo gốc	26
7	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	27
7.1	Kết luận	27
7.2	Hạn chế	27
7.3	Hướng phát triển	27
7.4	Bài học rút ra	28

TÓM TẮT

Đồ án này nghiên cứu khả năng ứng dụng SpikeGPT, một mô hình ngôn ngữ dựa trên mạng nơ-ron xung, cho bài toán chatbot agentic trong quản lý thương mại. Khác với các mô hình Transformer truyền thống, SpikeGPT được thiết kế theo hướng xử lý tuần tự, sử dụng nơ-ron Leaky Integrate-and-Fire, nhúng nhị phân, Spiking RWKV và Spiking Receptance Feed-Forward Network nhằm tiến gần hơn tới cơ chế tính toán thưa, tiết kiệm năng lượng của Spiking Neural Networks.

Nhóm xây dựng một bộ dữ liệu tool calling tiếng Việt cho miền thương mại, trong đó chatbot phải đọc yêu cầu của người dùng, chọn đúng công cụ quản lý và sinh ra lời gọi công cụ dưới dạng JSON. Sáu công cụ được xét gồm kiểm tra tồn kho, tạo đơn hàng, tra cứu đơn hàng, cập nhật đơn hàng, xóa đơn hàng và phân tích doanh thu. Để đánh giá mô hình, nhóm sử dụng các chỉ số Valid JSON, Intent Accuracy, Args Exact Match, Precision, Recall và F1-score.

Trên nền mô hình SpikeGPT-216M, nhóm thực nghiệm nhiều cấu hình: huấn luyện không thay đổi kiến trúc, huấn luyện từ đầu với HeadQK, fine-tuning với HeadQK, sửa dữ liệu, thử hàm mất mát có trọng số cho token cần copy và kiểm tra thêm ảnh hưởng của định dạng ID. Kết quả tốt nhất theo cân bằng tổng thể thuộc về SpikeGPT fine-tune với HeadQK ở epoch 220, đạt 99.80% Valid JSON, 94.10% Intent Accuracy và 13.94% Args Exact Match. Tuy nhiên, so với GPT-2 Small và GPT-2 Medium, SpikeGPT vẫn yếu rõ ở khả năng copy chính xác tham số, đặc biệt khi metric yêu cầu khớp toàn bộ arguments. Kết quả cho thấy SpikeGPT có thể học cấu trúc tool calling và định tuyến công cụ khá tốt, nhưng cơ chế hồi quy và biểu diễn xung nhị phân vẫn là hạn chế lớn đối với bài toán exact copying.

Từ khóa: SpikeGPT, Spiking Neural Networks, RWKV, Tool Calling, Agentic Chatbot, Fine-tuning, Exact Match.

Chương 1

TỔNG QUAN

1.1 Bối cảnh và lý do chọn đề tài

Các mô hình ngôn ngữ lớn hiện nay chủ yếu dựa trên kiến trúc Transformer. Nhờ cơ chế self-attention, Transformer có khả năng mô hình hóa quan hệ giữa các token rất mạnh và đạt hiệu quả cao trong nhiều tác vụ sinh ngôn ngữ, hỏi đáp, lập trình và hội thoại. Tuy nhiên, cơ chế này có chi phí tính toán lớn, đặc biệt do self-attention phải so sánh token với nhiều token khác trong chuỗi. Khi độ dài chuỗi tăng, chi phí bộ nhớ và tính toán tăng nhanh, gây khó khăn cho triển khai trên thiết bị biên hoặc các hệ thống có ràng buộc năng lượng.

Spiking Neural Networks (SNN) là một hướng tiếp cận khác, lấy cảm hứng từ cách nơ-ron sinh học truyền tín hiệu bằng xung rời rạc. Thay vì truyền các giá trị số thực liên tục ở mọi bước, SNN chỉ kích hoạt khi điện thế màng vượt ngưỡng, từ đó tạo ra tính thưa và tiềm năng tiết kiệm năng lượng trên phần cứng neuromorphic. Tuy vậy, việc kết hợp SNN với mô hình ngôn ngữ không đơn giản. Transformer yêu cầu softmax, phép nhân ma trận lớn và quan sát toàn cục chuỗi đầu vào; trong khi SNN vận hành tự nhiên theo từng bước thời gian với tín hiệu nhị phân.

Bài báo SpikeGPT đề xuất một hướng dung hòa: sử dụng kiến trúc RWKV thay cho self-attention toàn phần, sau đó xung hóa các thành phần chính để xây dựng mô hình ngôn ngữ sinh dựa trên SNN. Từ đó, nhóm chọn SpikeGPT làm nền tảng nghiên cứu cho bài toán chatbot agentic trong quản lý thương mại. Đây là bài toán phù hợp vì mô hình không chỉ cần sinh văn bản, mà còn phải hiểu ý định, chọn đúng công cụ và copy chính xác các tham số như mã đơn hàng, mã sản phẩm, số lượng, địa chỉ hoặc ngày tháng.

1.2 Mục tiêu đồ án

Đồ án đặt ra các mục tiêu chính sau:

1. Tìm hiểu bài báo SpikeGPT và các thành phần cốt lõi: LIF neuron, surrogate gradient, Binary Embedding, Spiking RWKV và SRFFN.

2. Xây dựng bài toán chatbot agentic tiếng Việt trong miền quản lý thương mại dưới dạng tool calling.
3. Tạo và tiền xử lý bộ dữ liệu huấn luyện, kiểm tra tính nhất quán giữa câu hỏi người dùng và arguments trong lời gọi công cụ.
4. Fine-tune SpikeGPT-216M và so sánh với các mô hình Transformer baseline như GPT-2 Small và GPT-2 Medium.
5. Thử một số cải tiến như HeadQK, sửa dữ liệu, copy-aware loss và kiểm tra ảnh hưởng của định dạng ID.
6. Phân tích nguyên nhân SpikeGPT vẫn kém Transformer ở Args Exact Match dù có thể đạt kết quả tốt ở Valid JSON và Intent Accuracy.

1.3 Phạm vi và đóng góp

Phạm vi của đề án không nhằm xây dựng một chatbot thương mại hoàn chỉnh để triển khai thực tế, mà tập trung vào năng lực cốt lõi của mô hình ngôn ngữ trong bối cảnh agentic: từ câu lệnh tự nhiên, mô hình phải sinh ra một lời gọi công cụ có cấu trúc. Nhóm không đánh giá chất lượng phản hồi hội thoại sau khi tool trả kết quả, mà tập trung vào ba tầng: cú pháp JSON, chọn đúng tool và sinh đúng arguments.

Đóng góp chính của nhóm gồm:

- Tái hiện và giải thích kiến trúc SpikeGPT trong ngữ cảnh SNN cho mô hình ngôn ngữ.
- Xây dựng dataset tool calling tiếng Việt cho quản lý thương mại.
- Thử nghiệm HeadQK trên SpikeGPT để hỗ trợ copy token từ ngữ cảnh.
- Xây dựng pipeline đánh giá với các metric Valid JSON, Intent Accuracy, Args Exact Match và F1-score.
- Thực hiện ablation study về dữ liệu, kiến trúc, hàm mất mát và định dạng ID.
- Phân tích giới hạn của SpikeGPT khi so sánh với Transformer trên bài toán exact argument copying.

1.4 Cấu trúc báo cáo

Phần còn lại của báo cáo được tổ chức theo hướng bám sát rubric của đề án cuối kỳ. Chương 2 trình bày các công trình liên quan, gồm Transformer, RWKV, SpikeGPT, SNN cho mô hình ngôn ngữ và tool calling. Chương 3 trình bày cơ sở lý thuyết cần thiết để hiểu SpikeGPT. Chương 4 mô tả dữ liệu và bài toán. Chương 5 trình bày phương pháp, mô hình, cải tiến và pipeline đánh giá. Chương 6 trình bày thực nghiệm, kết quả, ablation study và phân tích lỗi. Chương 7 kết luận, nêu hạn chế và hướng phát triển.

Chương 2

CÁC CÔNG TRÌNH LIÊN QUAN

2.1 Transformer và mô hình ngôn ngữ tự hồi quy

Transformer là kiến trúc nền tảng của phần lớn mô hình ngôn ngữ hiện đại. Điểm cốt lõi của Transformer là cơ chế self-attention, cho phép mỗi token truy cập trực tiếp các token khác trong chuỗi [3]. Với mô hình ngôn ngữ tự hồi quy như GPT-2, self-attention được kết hợp với causal mask để dự đoán token kế tiếp mà không nhìn thấy tương lai [4]. Lợi thế quan trọng của hướng này là khả năng liên kết thông tin ở khoảng cách xa, đặc biệt hữu ích khi mô hình cần copy một chuỗi xuất hiện trong prompt sang output.

Trong bài toán tool calling của nhóm, Transformer là baseline mạnh vì output thường phải lặp lại chính xác các thực thể trong input: mã đơn hàng, mã sản phẩm, số lượng, địa chỉ hoặc ngày tháng. Khi sinh token trong phần arguments, mô hình Transformer có thể dùng attention để đặt trọng số cao lên vị trí chứa thực thể gốc. Đây là lý do nhóm so sánh SpikeGPT với GPT-2 Small và GPT-2 Medium: không chỉ để so sánh điểm số, mà còn để kiểm tra liệu kiến trúc SNN/RWKV có thể thay thế attention đầy đủ trong một tác vụ cần exact copying hay không.

2.2 RWKV như một hướng thay thế attention

RWKV được đề xuất như một kiến trúc lai giữa RNN và Transformer [2]. Nó giữ tinh thần key-value và weighted aggregation của attention, nhưng triển khai theo dạng hồi quy tuyến tính theo thời gian. Thay vì lưu và xử lý toàn bộ ma trận attention kích thước $T \times T$, RWKV duy trì các trạng thái tích lũy và cập nhật chúng ở từng token. Cách làm này giúp giảm chi phí theo chiều dài chuỗi và tự nhiên phù hợp với sinh ngôn ngữ tự hồi quy.

Tuy nhiên, chính đặc điểm hồi quy cũng tạo ra khác biệt quan trọng. Transformer có thể truy cập trực tiếp token quá khứ qua attention score, còn RWKV phải nén lịch sử vào trạng thái. Nếu tác vụ yêu cầu hiểu ý tổng quát, trạng thái nén có thể đủ. Nếu tác vụ yêu cầu copy nguyên văn một chuỗi ID, việc nén có thể làm mất chi tiết. Báo cáo của nhóm khai thác đúng điểm này khi đưa SpikeGPT vào bài toán tool calling.

2.3 SpikeGPT và mô hình ngôn ngữ dựa trên SNN

SpikeGPT là công trình trung tâm của đề án. Bài báo *SpikeGPT: A Generative Pre-trained Language Model with Spiking Neural Networks* cho thấy mô hình ngôn ngữ sinh có thể được xây dựng trên nền SNN bằng cách kết hợp RWKV, Binary Embedding, LIF neuron và surrogate gradient [1]. So với nhiều SNN trước đó chủ yếu tập trung vào thị giác máy tính hoặc phân loại văn bản, SpikeGPT trực tiếp nhắm đến sinh ngôn ngữ tự nhiên, một bài toán khó hơn vì mô hình phải sinh chuỗi token dài và giữ ngữ cảnh.

Bài báo gốc đánh giá SpikeGPT trên các tác vụ NLG/NLU như Enwik8, WikiText và một số dataset phân loại văn bản. Kết quả cho thấy SpikeGPT có tiềm năng mở rộng và có thể đạt hiệu quả cạnh tranh trong một số thiết lập, đồng thời giữ lợi thế lý thuyết về tính thưa và tiêu thụ năng lượng. Tuy nhiên, bài báo gốc chưa tập trung vào tool calling, JSON generation hoặc exact argument copying. Đây là khoảng trống mà đề án của nhóm khảo sát.

2.4 Tool calling và agentic chatbot

Tool calling là một dạng bài toán trong đó mô hình ngôn ngữ không chỉ trả lời bằng văn bản mà phải sinh ra lời gọi hàm có cấu trúc. Một lời gọi tool thường gồm tên hàm và dictionary tham số. Trong hệ thống agentic, bước này đóng vai trò cầu nối giữa ngôn ngữ tự nhiên và hành động thực thi trên hệ thống bên ngoài. Sai tên tool có thể dẫn đến sai hành động; sai argument có thể dẫn đến truy vấn sai đơn hàng, cập nhật sai số lượng hoặc giao hàng sai địa chỉ.

So với phân loại intent truyền thống, tool calling khó hơn vì mô hình phải đồng thời giải hai bài toán: phân loại tool và sinh tham số. Bài toán thứ hai gần với sequence transduction có ràng buộc, trong đó một phần output phải được copy chính xác từ input. Vì vậy, nhóm sử dụng nhiều metric tách biệt: Valid JSON để đo cú pháp, Intent Accuracy để đo chọn tool, Args Exact Match để đo tham số, và F1-score để phân tích phân loại tool.

2.5 Vị trí của đề án so với các công trình trước

Đề án của nhóm khác bài báo SpikeGPT gốc ở ba điểm. Thứ nhất, thay vì đánh giá mô hình trên language modeling tổng quát, nhóm đưa SpikeGPT vào tác vụ agentic có cấu trúc. Thứ hai, thay vì chỉ tái hiện kiến trúc, nhóm thử cải tiến HeadQK và copy-aware loss để xử lý điểm yếu exact copying. Thứ ba, nhóm so sánh trực tiếp với GPT-2 để phân tích khoảng cách giữa SNN/RWKV và Transformer trên một bài toán có yêu cầu copy nghiêm ngặt.

Do đó, đóng góp của đề án không nằm ở việc đề xuất một kiến trúc hoàn toàn mới, mà ở việc kiểm tra một kiến trúc SNN ngôn ngữ mới trong một bối cảnh ứng dụng khó hơn, đồng thời phân tích rõ vì sao mô hình có thể làm tốt intent nhưng vẫn yếu ở arguments.

Chương 3

CƠ SỞ LÝ THUYẾT

3.1 Rào cản của Transformer đối với SNN

Transformer dựa trên self-attention, trong đó mỗi token có thể truy cập thông tin từ các token khác trong chuỗi. Với mô hình sinh ngôn ngữ, attention thường kết hợp causal mask để tránh rò rỉ thông tin tương lai. Cơ chế này rất hiệu quả trong việc copy, đối chiếu và liên kết thông tin xa, nhưng lại không phù hợp tự nhiên với SNN vì ba lý do.

Thứ nhất, self-attention xử lý toàn cục trên chuỗi, trong khi SNN hoạt động theo sự kiện và theo thời gian. Thứ hai, softmax cần phép tính số thực như hàm mũ và phép chia, không tương thích với mục tiêu tính toán nhị phân thưa của SNN. Thứ ba, attention có chi phí lớn theo độ dài chuỗi, khiến triển khai trên phần cứng tiết kiệm năng lượng trở nên khó khăn. Vì vậy, để xây dựng mô hình ngôn ngữ dựa trên SNN, cần một kiến trúc thay thế có tính tuần tự và chi phí tuyến tính hơn.

Về độ phức tạp, self-attention chuẩn với chuỗi dài T và chiều ẩn d cần tính ma trận tương quan giữa mọi cặp vị trí:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

Ma trận $QK^\top$ có kích thước $T \times T$, nên chi phí theo chiều dài chuỗi thường được mô tả là $O(T^2d)$. Trong khi đó, hướng RWKV/SRWKV cập nhật trạng thái theo từng bước thời gian, do đó có thể đạt chi phí tuyến tính $O(Td)$ theo chiều dài chuỗi. Đây là lý do SpikeGPT chọn RWKV làm xương sống thay vì đưa Transformer trực tiếp vào SNN.

Chèn hình so sánh self-attention toàn cục của Transformer và xử lý tuần tự của RWKV/SNN.

Hình 3.1: So sánh cơ chế xử lý chuỗi của Transformer và SpikeGPT

3.2 Spiking Neural Networks và LIF neuron

Spiking Neural Networks là thể hệ mạng nơ-ron sử dụng xung rời rạc để truyền thông tin. Một nơ-ron không phát tín hiệu liên tục ở mọi thời điểm, mà tích lũy điện thế màng theo thời gian. Khi điện thế vượt ngưỡng, nơ-ron phát xung và trạng thái được reset. Mô hình Leaky Integrate-and-Fire (LIF) thường được dùng để mô tả quá trình này.

Một dạng rời rạc đơn giản của LIF có thể viết như sau:

$$u_t = \lambda u_{t-1} + x_t - v_{th} s_{t-1}$$

$$s_t = H(u_t - v_{th})$$

Trong đó u_t là điện thế màng tại thời điểm t , λ là hệ số rò rỉ, x_t là tín hiệu đầu vào, v_{th} là ngưỡng phát xung, s_t là xung đầu ra và $H(\cdot)$ là hàm Heaviside. Nếu điện thế vượt ngưỡng, nơ-ron phát ra xung nhị phân. Nếu không, đầu ra bằng 0.

Khó khăn chính của LIF là hàm Heaviside không khả vi. Trong huấn luyện bằng lan truyền ngược, đạo hàm của hàm bước gần như bằng 0 ở hầu hết mọi nơi, làm gradient không thể truyền hiệu quả. SpikeGPT giải quyết bằng surrogate gradient: trong forward vẫn dùng xung nhị phân, nhưng trong backward thay đạo hàm của Heaviside bằng một hàm xấp xỉ trơn, chẳng hạn arctangent surrogate.

Có thể hiểu quá trình này như một thủ thuật tách biệt giữa lan truyền xuôi và lan truyền ngược. Ở forward, mô hình vẫn giữ tính chất xung:

$$s_t = H(u_t - v_{th})$$

Nhưng khi tính gradient, đạo hàm không dùng trực tiếp $H'(x)$, mà thay bằng một hàm surrogate $\sigma'(x)$:

$$\frac{\partial s_t}{\partial u_t} \approx \sigma'(u_t - v_{th})$$

Nhờ vậy, mạng vẫn có thể được huấn luyện bằng backpropagation trong khi đầu ra trung gian vẫn là spike rời rạc. Đây là điểm then chốt giúp SNN có thể mở rộng sang các tác vụ học sâu phức tạp như mô hình ngôn ngữ.

Chèn hình minh họa điện thế màng tích lũy, vượt ngưỡng, phát xung và reset của LIF neuron.

Hình 3.2: Động học của nơ-ron LIF

3.3 Kiến trúc SpikeGPT

SpikeGPT là mô hình ngôn ngữ sinh kết hợp giữa đặc tính hồi quy của RWKV và cơ chế nơ-ron xung của SNN. Thay vì sử dụng self-attention như Transformer, SpikeGPT dùng Spiking RWKV để xử lý thông tin theo từng bước thời gian. Điều này giúp mô hình phù hợp hơn với cơ chế tuần tự của SNN và có độ phức tạp tuyến tính theo chiều dài chuỗi.

3.3.1 Binary Embedding

Ở các mô hình ngôn ngữ truyền thống, token ID được ánh xạ sang vector embedding số thực. Với SpikeGPT, để phù hợp với SNN, embedding được đưa qua cơ chế xung hóa để tạo biểu diễn nhị phân. Trong forward pass, hàm Heaviside ép đầu ra thành xung 0/1. Trong backward pass, surrogate gradient cho phép cập nhật trọng số embedding liên tục. Nhờ vậy, mô hình vẫn có thể học bằng gradient descent nhưng duy trì tính chất spiking khi lan truyền xuôi.

3.3.2 Spiking RWKV

RWKV có thể được xem như một kiến trúc lai giữa Transformer và RNN. Nó giữ ý tưởng key-value và cơ chế trọng số theo thời gian, nhưng xử lý tuần tự thay vì tính attention toàn cục. Tại mỗi bước thời gian, mô hình tính các thành phần receptance, key và value, đồng thời duy trì trạng thái ẩn tích lũy từ quá khứ. Cơ chế decay giúp mô hình quyết định thông tin cũ nên được giữ lại hay suy giảm.

Trong SpikeGPT, RWKV được chuyển thành Spiking RWKV. Dữ liệu được xử lý theo từng token $X[t]$, phù hợp với SNN. Thay vì tạo ma trận attention lớn, SRWKV sử dụng các phép toán theo phần tử và trạng thái hồi quy. Điều này giảm chi phí tính toán, đồng thời tránh nhu cầu dùng causal mask như Transformer vì bản chất tuần tự đã đảm bảo mô hình không nhìn thấy tương lai.

Một cách mô tả đơn giản hóa của RWKV tại thời điểm t là:

$$r_t = \sigma(W_r x_t), \quad k_t = W_k x_t, \quad v_t = W_v x_t$$

$$a_t = e^{-w} \odot a_{t-1} + e^{k_t} \odot v_t$$

$$b_t = e^{-w} \odot b_{t-1} + e^{k_t}$$

$$o_t = r_t \odot \frac{a_t}{b_t}$$

Trong đó r_t đóng vai trò cổng receptance, k_t và v_t lần lượt tương tự key và value, a_t , b_t là hai trạng thái tích lũy quá khứ, w là tham số decay học được, và $\odot$ là phép nhân từng phần.

tử. Công thức này cho thấy RWKV không cần lưu toàn bộ ma trận attention $T \times T$, mà chỉ cần cập nhật trạng thái theo thời gian. Khi đưa vào SpikeGPT, các tín hiệu này tiếp tục được xử lý qua cơ chế spiking để phù hợp hơn với SNN.

Chèn sơ đồ một block SpikeGPT gồm Binary Embedding, SRWKV, SRFFN, residual connection và readout layer.

Hình 3.3: Kiến trúc tổng quát của một block SpikeGPT

3.3.3 Spiking Receptance Feed-Forward Network

SRFFN là phiên bản xung hóa của khối feed-forward có cơ chế gating. Một nhánh tạo cổng điều tiết lượng thông tin được phép đi qua, nhánh còn lại xử lý nội dung chính. Đầu ra tiếp tục được đưa qua nơ-ron LIF để tạo tính thưa. Theo bài báo SpikeGPT, SRFFN giữ vai trò quan trọng vì các lớp feed-forward trong mô hình ngôn ngữ thường chứa nhiều thông tin ngữ nghĩa và tri thức nội tại.

Dạng khái quát của một khối feed-forward có gating có thể viết:

$$g_t = \sigma(W_g x_t), \quad h_t = \phi(W_h x_t)$$

$$y_t = \text{LIF}(g_t \odot h_t)$$

Trong đó g_t là cổng kiểm soát lượng thông tin đi qua, h_t là nhánh xử lý nội dung và ϕ là hàm kích hoạt phi tuyến. Việc đưa LIF vào sau phép trộn kênh giúp đầu ra trở thành xung thưa, nhưng đồng thời cũng khiến thông tin phải đi qua một bước lượng tử hóa mạnh hơn so với feed-forward số thực của Transformer.

3.4 Ưu điểm và giới hạn kỳ vọng

SpikeGPT hướng tới ba ưu điểm chính: xử lý chuỗi với chi phí tuyến tính, tương thích hơn với phần cứng neuromorphic và giảm năng lượng lý thuyết nhờ spike thưa. Tuy nhiên, chính các đặc điểm này cũng tạo ra giới hạn. Khi thông tin được nén qua trạng thái hồi quy và xung nhị phân, mô hình có thể mất chi tiết token-level, đặc biệt với các tác vụ yêu cầu copy chính xác từng ký tự như mã đơn hàng hoặc địa chỉ. Đây là giả thuyết quan trọng được kiểm chứng trong phần thực nghiệm của nhóm.

Chương 4

BÀI TOÁN VÀ DỮ LIỆU

4.1 Bài toán chatbot agentic trong thương mại

Trong đồ án, chatbot không chỉ trả lời bằng ngôn ngữ tự nhiên mà phải đóng vai trò agent. Khi nhận yêu cầu từ người quản lý, agent cần xác định tác vụ, trích xuất tham số và sinh ra lời gọi công cụ đúng định dạng. Lời gọi công cụ được đặt trong cặp thẻ đặc biệt:

```
<tool_call>
{"name": "ten_tool", "arguments": {...}}
</tool_call>
```

Bài toán này có tính thực tế cao vì nhiều hệ thống thương mại hiện đại cần agent tự động tương tác với API: kiểm kho, tạo đơn, cập nhật đơn, xóa đơn hoặc tổng hợp doanh thu. Đối với mô hình ngôn ngữ, đây là bài toán khó hơn sinh câu trả lời tự do vì output phải đúng cả cú pháp lẫn nội dung tham số.

Về mặt hình thức, mỗi mẫu dữ liệu có thể xem là một cặp (x, z) , trong đó x là câu yêu cầu tiếng Việt của người dùng, còn z là tool call chuẩn:

$$z = (n, A)$$

Ở đây $n \in \mathcal{N}$ là tên tool thuộc tập sáu tool đã định nghĩa, còn $A = \{(k_i, v_i)\}_{i=1}^m$ là tập arguments. Mục tiêu của mô hình là sinh chuỗi token biểu diễn z sao cho:

$$\hat{z} = \arg \max_z P_\theta(z | x)$$

Khác với phân loại intent đơn thuần, bài toán này gồm hai tầng phụ thuộc nhau. Tầng thứ nhất là chọn đúng n . Tầng thứ hai là sinh đúng từng khóa và từng giá trị trong A . Nếu tầng thứ nhất đúng nhưng tầng thứ hai sai, hệ thống vẫn có thể gọi API sai và gây lỗi nghiệp vụ.

Chèn hình pipeline: User request -> SpikeGPT -> JSON tool call -> API thương mại.

Hình 4.1: Pipeline bài toán chatbot agentic trong quản lý thương mại

4.2 Bộ công cụ

Nhóm định nghĩa sáu công cụ chính:

Bảng 4.1: Các tool trong bài toán agentic chatbot

Tool	Ý nghĩa
check_inventory	Kiểm tra tồn kho của một sản phẩm dựa trên product_id.
create_order	Tạo đơn hàng mới với sản phẩm, số lượng và địa chỉ giao hàng.
get_order	Tra cứu thông tin đơn hàng theo order_id.
update_order	Cập nhật đơn hàng, thường gồm mã đơn, số lượng hoặc địa chỉ mới.
delete_order	Hủy hoặc xóa đơn hàng theo order_id.
revenue_analysis	Phân tích doanh thu theo khoảng thời gian.

Bảng 4.2: Schema arguments của từng tool

Tool	Arguments chính
check_inventory	product_id: mã sản phẩm cần kiểm tra tồn kho.
create_order	product_id, quantity, address: sản phẩm, số lượng và địa chỉ giao hàng.
get_order	order_id: mã đơn hàng cần tra cứu.
update_order	order_id và có thể thêm quantity, address tùy yêu cầu cập nhật.
delete_order	order_id: mã đơn hàng cần hủy.
revenue_analysis	start_date, end_date: khoảng thời gian phân tích doanh thu.

4.3 Xây dựng dữ liệu

Vì chưa có sẵn bộ dữ liệu tool calling tiếng Việt chuyên cho miền thương mại, nhóm tự xây dựng dữ liệu. Quy trình gồm: định nghĩa schema tool, viết các mẫu hội thoại, sử dụng mô hình mạnh như Gemini hoặc GPT để sinh thêm câu lệnh tự nhiên, sau đó parse lại thành định dạng JSONL phù hợp với SpikeGPT. Mỗi mẫu gồm phần system prompt, câu người dùng và lời gọi tool của assistant.

Một ví dụ rút gọn:

```

<|im_start|>user
Kim tra otn kho asn apbm Airpods5338 giúp tôi.
<|im_end|>
<|im_start|>assistant
<tool_call>
{"name": "check_inventory",
 "arguments": {"product_id": "Airpods5338"}}
</tool_call>
<|im_end|>

```

4.4 Tiền xử lý và kiểm tra chất lượng

Sau giai đoạn sinh dữ liệu ban đầu, nhóm phát hiện nhiều mẫu có vấn đề: tham số trong arguments không xuất hiện đúng trong câu người dùng, giá trị None/null, hoặc khác biệt nhỏ về dấu gạch nối. Điều này đặc biệt nguy hiểm với bài toán copy, vì mô hình sẽ học từ tín hiệu sai: câu hỏi chứa một giá trị nhưng nhãn yêu cầu sinh giá trị khác.

Nhóm viết script kiểm tra và sửa dữ liệu theo hướng bảo toàn copy signal. Các mẫu có thể sửa được sẽ thay biến thể trong câu user thành đúng giá trị trong nhãn; các mẫu không thể sửa chắc chắn bị loại bỏ. Sau xử lý, bộ train copyfix còn 50,249 mẫu và bộ validation copyfix còn 6,730 mẫu, với mục tiêu không còn argument literal bị thiếu trong câu người dùng.

Quy tắc kiểm tra chất lượng quan trọng nhất là: với những argument cần copy, giá trị trong nhãn phải xuất hiện nguyên văn trong câu người dùng. Nếu ký hiệu câu người dùng là x và một argument là (k, v) , điều kiện mong muốn là:

$$v \in x, \quad k \in \{\text{order_id, product_id, quantity, address, start_date, end_date}\}$$

Nếu điều kiện này không đúng, mẫu đó tạo ra tín hiệu học mâu thuẫn: mô hình được yêu cầu sinh một giá trị không có trong input. Với bài toán copy, đây là lỗi nghiêm trọng hơn lỗi diễn đạt tự nhiên, vì nó trực tiếp làm giảm khả năng học ánh xạ từ input sang arguments.

Bảng 4.3: Kết quả làm sạch dữ liệu copyfix

Tập dữ liệu	Số mẫu sau xử lý	Mục tiêu kiểm tra
Train copyfix	50,249	Không còn argument literal bị thiếu
Validation copyfix	6,730	Không còn argument literal bị thiếu

4.5 Định dạng ID và phân phối dữ liệu

Trong quá trình phân tích lỗi, nhóm kiểm tra thêm phân phối định dạng ID. Dữ liệu train sau copyfix có `order_id` chủ yếu ở dạng chữ nối số, ví dụ DH9200, chiếm khoảng 81.2%. Với `product_id`, dạng chữ nối số chiếm khoảng 75.3%. Trong khi đó, một bộ test OOD ban đầu sử dụng 100% `order_id` dạng chữ-gạch-số và 100% `product_id` dạng nhiều segment có gạch nối như MAC-59-PR0. Nhóm tạo thêm một bộ test in-distribution bằng cách chuẩn hóa ID về dạng gần train hơn. Kết quả không cải thiện đáng kể, cho thấy định dạng ID không phải nguyên nhân chính làm Args Exact Match thấp; điểm nghẽn nằm sâu hơn ở năng lực exact copying của mô hình.

Chương 5

PHƯƠNG PHÁP THỰC NGHIỆM

5.1 Các mô hình được đánh giá

Nhóm đánh giá hai nhóm mô hình. Nhóm thứ nhất là SpikeGPT-216M, gồm các biến thể không dùng HeadQK, huấn luyện từ đầu với HeadQK và fine-tuning từ checkpoint pre-trained. Nhóm thứ hai là các baseline Transformer gồm GPT-2 Small và GPT-2 Medium sau fine-tuning trên cùng bài toán.

Việc so sánh với GPT-2 có ý nghĩa vì GPT-2 là Transformer autoregressive có cơ chế self-attention đầy đủ. Nếu GPT-2 vượt trội ở Args Exact Match, điều này cho thấy attention vẫn là lợi thế lớn trong bài toán copy tham số từ input sang output.

5.2 Nguồn cài đặt và các chỉnh sửa chính

Phần cài đặt SpikeGPT của nhóm dựa trên mã nguồn mở của tác giả bài báo SpikeGPT [6]. Nhóm sử dụng lại các thành phần nền tảng như kiến trúc GPT/RWKV, tokenizer, CUDA kernel cho WKV và pipeline huấn luyện. Trên nền đó, nhóm chỉnh sửa các file huấn luyện và mô hình để phù hợp với bài toán tool calling tiếng Việt.

Các chỉnh sửa quan trọng gồm:

- Thay dữ liệu language modeling tổng quát bằng dữ liệu hội thoại tool calling theo định dạng chat template.
- Bật và cấu hình HeadQK với kích thước 256 chiều để hỗ trợ copy token.
- Chỉnh đường dẫn dữ liệu train/validation sang bộ copyfix đã làm sạch.
- Thêm copy-aware loss trong `model.py` để tăng trọng số cho token từng xuất hiện trong ngữ cảnh.
- Xây dựng notebook đánh giá riêng để parse output, kiểm tra JSON, so sánh intent và so sánh arguments.

Việc nêu rõ nguồn cài đặt là cần thiết vì đồ án không viết lại toàn bộ SpikeGPT từ đầu. Phần đóng góp của nhóm nằm ở việc tái sử dụng có kiểm soát một implementation nghiên

cứu, đưa nó sang miền dữ liệu mới, thêm cải tiến phù hợp với bài toán và đánh giá bằng benchmark riêng.

Bảng 5.1: Cấu hình chính của các nhóm mô hình

Mô hình	Quy mô	Vai trò trong thực nghiệm
SpikeGPT	216M tham số	Mô hình SNN/RWKV chính, kiểm tra khả năng tool calling trên kiến trúc spiking.
SpikeGPT + HeadQK	216M + head phụ	Biến thể cải tiến để tăng khả năng đối chiếu và copy token từ ngữ cảnh.
GPT-2 Small	124M tham số	Baseline Transformer nhỏ, dùng để so sánh ảnh hưởng của self-attention.
GPT-2 Medium	355M tham số	Baseline Transformer lớn hơn, kiểm tra trần hiệu năng của hướng Transformer trong cùng bài toán.

5.3 Thiết lập huấn luyện

Với SpikeGPT, nhóm sử dụng cấu hình 216M tham số gồm 18 layer, kích thước embedding 768 và context length 1024. Mô hình được huấn luyện theo Causal Language Modeling, tức tại mỗi vị trí mô hình dự đoán token kế tiếp. Batch size thực nghiệm bị giới hạn bởi tài nguyên GPU, nên quá trình huấn luyện cần nhiều epoch để hội tụ. Các thí nghiệm dài nhất của nhóm kéo dài hàng trăm epoch và tiêu tốn hàng chục giờ cho mỗi cấu hình.

Bảng 5.2: Một số thiết lập huấn luyện chính

Thành phần	Thiết lập
Backbone SpikeGPT	216M tham số, 18 layer, embedding dimension 768.
Context length	1024 token.
HeadQK	256 chiều trong các cấu hình cải tiến.
Loss gốc	Causal Language Modeling với Cross-Entropy Loss.
Loss thử nghiệm	Copy-aware weighted Cross-Entropy, $\alpha = 2.0$, copy window 512 token.
Dữ liệu chính	Train copyfix 50,249 mẫu, validation copyfix 6,730 mẫu.
Baseline	GPT-2 Small và GPT-2 Medium fine-tuned trên cùng dạng dữ liệu tool calling.

Nhóm đánh giá hai chiến lược khởi tạo. Với *training from scratch*, toàn bộ mô hình được học lại từ đầu trên dữ liệu tool calling. Với *fine-tuning*, mô hình bắt đầu từ checkpoint SpikeGPT-216M pretrained rồi tiếp tục học trên dữ liệu của nhóm. Kết quả cho thấy fine-tuning ổn định hơn đáng kể vì mô hình đã có sẵn năng lực ngôn ngữ nền, còn training from

scratch trên dữ liệu nhỏ dễ bị overfit hoặc học kém cấu trúc ngôn ngữ.

5.4 HeadQK

HeadQK là cải tiến kiến trúc quan trọng nhất nhóm sử dụng cho SpikeGPT. Ý tưởng là thêm một cơ chế tương tự attention nhẹ ở phần đầu ra, với kích thước 256 chiều. Mục tiêu không phải thay thế toàn bộ SRWKV bằng Transformer, mà cung cấp cho mô hình một khả năng đối chiếu lại ngữ cảnh gần hơn khi cần sinh token cụ thể trong arguments.

Về mặt đánh đổi, HeadQK làm mô hình nặng hơn và giảm phần nào tinh thần tiết kiệm của SpikeGPT. Tuy nhiên, với bài toán tool calling, yêu cầu copy chính xác tham số là rất quan trọng. Do đó, nhóm chấp nhận hy sinh một phần độ nhẹ để tăng khả năng sinh đúng arguments.

Có thể mô tả trực giác của HeadQK như một lớp đối chiếu bổ sung trên hidden states $H = [h_1, \dots, h_T]$. Từ hidden states, mô hình tạo query và key:

$$Q = HW_Q, \quad K = HW_K$$

Sau đó tính điểm tương đồng nhân quả giữa vị trí hiện tại và các vị trí trước:

$$S = \frac{QK^\top}{\sqrt{d_q}} + M_{\text{causal}}$$

$$A = \text{softmax}(S)$$

Trong đó $d_q = 256$ là kích thước HeadQK trong thực nghiệm, còn M_{causal} là mặt nạ nhân quả để không nhìn tương lai. Khi sinh token trong phần arguments, lớp này giúp mô hình có thêm tín hiệu rằng token cần sinh có thể đã xuất hiện trong prompt. Vì vậy, HeadQK được kỳ vọng hỗ trợ các trường như `order_id` và `product_id`. Tuy nhiên, do chỉ là một head phụ ở đầu ra, nó không có khả năng thay thế hoàn toàn multi-head self-attention nhiều tầng của GPT-2.

Chèn hình minh họa HeadQK: hidden states -> Q/K -> attention score -> hỗ trợ copy token trong arguments.

Hình 5.1: Cơ chế HeadQK được thêm vào SpikeGPT

5.5 Hàm mất mát

Thiết lập huấn luyện cơ bản dùng Causal Language Modeling. Tại mỗi vị trí, mô hình dự đoán token kế tiếp và tối ưu cross-entropy:

$$\mathcal{L}_{\text{CE}} = - \sum_{t=1}^T \sum_{c=1}^V y_{t,c} \log(\hat{y}_{t,c})$$

Trong đó T là chiều dài chuỗi, V là kích thước từ vựng, $y_{t,c}$ là nhãn one-hot và $\hat{y}_{t,c}$ là xác suất dự đoán sau softmax ở readout layer. Cross-entropy đối xử gần như ngang nhau với mọi token, trong khi bài toán tool calling lại đặc biệt nhạy với token thuộc arguments. Sai một token trong mã đơn hoặc địa chỉ có thể làm toàn bộ Args Exact Match bị tính sai.

Vì vậy, nhóm thử thêm copy-aware weighted loss:

$$\mathcal{L}_{\text{copy}} = \frac{\sum_{t=1}^T w_t \cdot \text{CE}(\hat{y}_t, y_t)}{\sum_{t=1}^T w_t}$$

$$w_t = \begin{cases} \alpha, & \text{nếu token mục tiêu đã xuất hiện trong ngữ cảnh trước đó} \\ 1, & \text{ngược lại} \end{cases}$$

Trong thực nghiệm, nhóm dùng $\alpha = 2.0$ và cửa sổ copy 512 token. Ý tưởng là tăng trọng số cho các token có khả năng cần copy từ câu hỏi người dùng, như `order_id`, `product_id`, số lượng hoặc một phần địa chỉ. Kết quả cho thấy Args Exact Match tăng từ 13.94% lên 16.88%, nhưng Intent Accuracy và F1-score giảm. Do đó, nhóm không chọn loss mới làm mô hình chính, mà xem đây là một ablation chứng minh rằng việc nhấn mạnh token copy có tác dụng cục bộ nhưng làm mất cân bằng học intent.

Điểm cần nhấn mạnh là loss mới không thay đổi bản chất bài toán Causal Language Modeling. Mô hình vẫn học dự đoán token kế tiếp. Thay đổi duy nhất nằm ở trọng số từng vị trí token. Nếu token mục tiêu y_t đã xuất hiện trong cửa sổ ngữ cảnh trước đó:

$$\exists j < t, \quad t - j \leq 512, \quad x_j = y_t$$

thì vị trí đó được xem là copy-sensitive và có trọng số lớn hơn. Cách làm này dễ giải thích, không cần thay đổi output format, nhưng có rủi ro: mô hình có thể quá tập trung vào token copy và giảm năng lực phân biệt intent tổng quát.

5.6 Các chỉ số đánh giá

Bảng 5.3: Các metric trong pipeline đánh giá

Metric	Ý nghĩa
Valid JSON	Tỷ lệ model sinh được output đúng cú pháp JSON trong thẻ <code><tool_call></code> .
Intent Accuracy	Tỷ lệ model chọn đúng tên công cụ cần gọi.
Args Exact Match	Tỷ lệ model sinh đúng toàn bộ dictionary arguments so với ground truth.
Precision	Khi model dự đoán một tool, tỷ lệ dự đoán đó thật sự đúng.
Recall	Trong các mẫu thật sự thuộc một tool, tỷ lệ model nhận ra đúng tool đó.
F1-score	Trung bình điều hòa giữa Precision và Recall, phản ánh chất lượng phân loại tool.

Cần lưu ý rằng Args Exact Match trong notebook đánh giá so sánh toàn bộ dictionary arguments, không chỉ riêng ID. Vì vậy, nếu sai một ký tự trong địa chỉ, sai kiểu dữ liệu của số lượng hoặc thừa thiếu một khóa, mẫu đó bị tính sai hoàn toàn.

Các metric phân loại tool được tính từ ma trận nhầm lẫn giữa tool thật và tool dự đoán. Với một tool c , ký hiệu TP_c là số mẫu tool c được dự đoán đúng, FP_c là số mẫu không thuộc c nhưng bị dự đoán thành c , và FN_c là số mẫu thuộc c nhưng bị dự đoán sang tool khác. Khi đó:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}$$

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c}$$

$$\text{F1}_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}$$

Intent Accuracy là accuracy trên tên tool:

$$\text{IntentAcc} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{n}_i = n_i]$$

Args Exact Match là tỷ lệ toàn bộ dictionary arguments khớp tuyệt đối:

$$\text{ArgsEM} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{A}_i = A_i]$$

Weighted F1 là trung bình F1 theo số lượng mẫu của từng tool:

$$\text{WeightedF1} = \sum_{c \in \mathcal{N}} \frac{N_c}{N} \text{F1}_c$$

Vì dataset test tương đối cân bằng giữa các tool, Weighted F1 thường gần với Intent Accuracy. Tuy nhiên, F1 vẫn hữu ích hơn khi phân tích sâu vì nó chỉ ra tool nào có precision thấp, tool nào có recall thấp.

Chèn hình confusion matrix chọn tool từ notebook đánh giá.

Hình 5.2: Ma trận nhầm lẫn cho bài toán chọn tool

5.7 Thiết kế ablation

Nhóm thực hiện các hướng thử nghiệm sau:

- **Chỉ đổi dữ liệu:** thêm phần suy nghĩ [THOUGHT] vào prompt để hy vọng mô hình ghi nhớ ID tốt hơn. Kết quả thất bại vì chuỗi dài hơn làm mô hình dễ lộn và sinh sai cấu trúc.
- **Chỉ đổi kiến trúc:** thêm HeadQK để hỗ trợ copy token. Kết quả cải thiện format JSON và Args Exact Match nhưng chưa giải quyết triệt để.
- **Kết hợp dữ liệu và kiến trúc:** thêm cả [THOUGHT] và HeadQK. Kết quả xấu do nhiễu giữa phần sinh tiếng Việt và phần sinh JSON.
- **Gating, freeze và local window:** thử các kỹ thuật giảm sóc khi thêm attention. Kết quả vẫn không ổn định, cho thấy lỗi SpikeGPT 216M có giới hạn dung lượng đối với tác vụ exact copying.
- **Sửa dữ liệu và copy-aware loss:** cải thiện chất lượng copy signal và tăng trọng số token cần copy. Kết quả tăng Args nhưng giảm intent/F1.
- **Kiểm tra định dạng ID:** tạo thêm test set chuẩn hóa ID về dạng gần train. Kết quả không khác biệt lớn, loại trừ giả thuyết ID format là nguyên nhân chính.

5.8 Pipeline suy luận và đánh giá

Pipeline đánh giá được thiết kế để đo đúng năng lực tool calling, không chỉ đo loss huấn luyện. Với mỗi mẫu test, hệ thống lấy câu user prompt, ghép với system prompt cố định,

sau đó yêu cầu mô hình sinh tiếp phần assistant. Output sinh ra được đưa qua bộ parser để tìm đoạn `<tool_call>` và parse JSON.

Quy trình gồm các bước:

1. Đọc ground truth từ JSONL, gồm câu user, tên tool thật và arguments thật.
2. Tạo prompt suy luận theo cùng chat template khi train.
3. Sinh output bằng greedy decoding cho đến khi gặp `</tool_call>` hoặc đạt giới hạn token.
4. Trích xuất JSON từ output; nếu parse thất bại, gán nhãn `invalid_json` hoặc `no_json_found`.
5. So sánh name dự đoán với name thật để tính Intent Accuracy và các metric phân loại.
6. So sánh nguyên dictionary arguments dự đoán với ground truth để tính Args Exact Match.
7. Lưu toàn bộ prediction, raw output và trạng thái đúng/sai để phục vụ phân tích lỗi.

Cách đánh giá này nghiêm ngặt hơn việc chỉ nhìn vài mẫu demo. Một output nhìn bằng mắt có thể “gần đúng”, nhưng nếu JSON không parse được hoặc arguments sai một trường, hệ thống tự động vẫn tính là lỗi. Điều này phù hợp với bối cảnh API calling thực tế, nơi định dạng và tham số cần chính xác.

Chương 6

KẾT QUẢ VÀ PHÂN TÍCH

6.1 Kết quả tổng quan

Bảng 6.1: So sánh SpikeGPT và GPT-2 trên bài toán tool calling

Mô hình	Valid JSON	Intent Acc	Args EM	Precision	Recall	F1
SpikeGPT NoHeadQK ep78	100.00	86.62	0.00	0.8781	0.8662	0.8659
SpikeGPT Scratch HeadQK ep78	95.98	84.64	1.78	0.8944	0.8464	0.8655
SpikeGPT Scratch HeadQK ep220	91.52	81.24	4.68	0.9022	0.8124	0.8463
SpikeGPT Finetune HeadQK ep220	99.80	94.10	13.94	0.9459	0.9410	0.9415
GPT-2 Small	99.98	96.52	68.30	0.9671	0.9652	0.9648
GPT-2 Medium	100.00	96.78	67.76	0.9696	0.9678	0.9678

Bảng kết quả cho thấy SpikeGPT có thể học tốt cấu trúc tool calling ở hai tầng đầu. Mô hình fine-tune với HeadQK đạt 99.80% Valid JSON và 94.10% Intent Accuracy. Điều này chứng minh mô hình có khả năng sinh JSON hợp lệ và hiểu tương đối tốt yêu cầu của người dùng để chọn tool phù hợp.

Tuy nhiên, Args Exact Match vẫn là điểm nghẽn lớn. Cấu hình SpikeGPT tốt nhất chỉ đạt 13.94%, trong khi GPT-2 Small và GPT-2 Medium đạt khoảng 68%. Chênh lệch này rất lớn và nhất quán với trực giác: Transformer có self-attention đầy đủ nên thuận lợi hơn trong việc copy thông tin cụ thể từ input sang output.

Khi đọc bảng kết quả, cần tách ba câu hỏi khác nhau. Thứ nhất, mô hình có sinh được JSON đúng cú pháp hay không; SpikeGPT làm khá tốt ở tầng này. Thứ hai, mô hình có hiểu tác vụ hay không; SpikeGPT fine-tune cũng đạt mức cao. Thứ ba, mô hình có copy đúng toàn bộ arguments hay không; đây là nơi SpikeGPT thua rõ Transformer. Do đó, kết luận không phải là SpikeGPT “không học được”, mà là SpikeGPT học được format và intent nhưng chưa đủ mạnh cho exact copying.

Chèn biểu đồ cột so sánh Valid JSON, Intent Accuracy, Args Exact Match và F1 giữa SpikeGPT và GPT-2.

Hình 6.1: So sánh tổng quan các metric chính

6.2 Phân tích theo từng nhóm metric

Valid JSON. Tỷ lệ JSON hợp lệ cao cho thấy mô hình học được khung output. Với mô hình Finetune HeadQK ep220, 99.80% output parse được thành JSON. Điều này quan trọng vì nếu JSON sai cú pháp, hệ thống phía sau không thể gọi tool dù intent có thể đúng.

Intent Accuracy và F1-score. SpikeGPT fine-tune đạt 94.10% Intent Accuracy và F1-score 0.9415. Hai chỉ số này gần nhau vì tập test tương đối cân bằng giữa các tool. Kết quả cho thấy mô hình đã học được ngữ nghĩa tổng quát của câu lệnh, ví dụ phân biệt kiểm kho, tạo đơn, cập nhật đơn và phân tích doanh thu.

Args Exact Match. Đây là chỉ số thấp nhất và cũng là chỉ số quan trọng nhất cho tool calling thực tế. SpikeGPT fine-tune chỉ đạt 13.94%, trong khi GPT-2 đạt khoảng 68%. Khoảng cách này cho thấy bài toán không dừng ở phân loại intent. Mô hình phải thực hiện một dạng copy có điều kiện, trong đó token cụ thể ở input cần được tái hiện chính xác ở output.

Precision và Recall. Precision trả lời câu hỏi: khi mô hình chọn một tool, lựa chọn đó đáng tin đến mức nào. Recall trả lời câu hỏi: với các mẫu thật sự thuộc một tool, mô hình bắt được bao nhiêu. Trong báo cáo kết quả, một số tool có precision cao nhưng recall thấp, nghĩa là mô hình chỉ chọn tool đó khi khá chắc nhưng bỏ sót nhiều mẫu. Ngược lại, tool có recall cao nhưng precision thấp thường bị mô hình chọn quá tay.

6.3 Tác động của HeadQK

Khi không dùng HeadQK, SpikeGPT đạt 0.00% Args Exact Match dù Valid JSON và Intent Accuracy vẫn khá cao. Điều này cho thấy mô hình có thể học khuôn dạng tổng quát nhưng gần như không copy đúng tham số hoàn chỉnh. Khi thêm HeadQK, Args Exact Match tăng lên 1.78% ở epoch 78 và 4.68% ở epoch 220 khi train from scratch. Với fine-tuning từ checkpoint pretrained, HeadQK giúp mô hình đạt 13.94% Args Exact Match.

Kết quả này cho thấy HeadQK là cải tiến có tác dụng thật, nhưng chưa đủ mạnh. Nó giúp mô hình giảm lỗi format và tăng khả năng đối chiếu token, song không thể thay thế hoàn toàn cơ chế attention nhiều lớp của Transformer. Đây là điểm quan trọng khi bảo vệ đồ án: nhóm không chỉ tái hiện paper mà đã thử điều chỉnh kiến trúc theo nhu cầu bài toán, đồng thời phân tích rõ đánh đổi.

Nếu xem NoHeadQK là baseline SpikeGPT thuần, việc Args Exact Match tăng từ

0.00% lên 13.94% ở cấu hình fine-tune tốt nhất cho thấy HeadQK không phải thay đổi hình thức. Nó trực tiếp tác động đến metric yếu nhất. Tuy vậy, mức 13.94% vẫn thấp hơn rất nhiều so với GPT-2. Điều này cho phép rút ra kết luận cân bằng: HeadQK có cải thiện, nhưng chưa giải quyết triệt để giới hạn copy của kiến trúc SNN/RWKV.

Bảng 6.2: Tóm tắt tác động của các cấu hình SpikeGPT

Cấu hình	Nhận xét chính
NoHeadQK ep78	Học được JSON và intent ở mức khá, nhưng Args Exact Match bằng 0, cho thấy gần như không copy hoàn chỉnh.
Scratch HeadQK ep78	HeadQK bắt đầu giúp Args EM khác 0, nhưng mô hình chưa đủ hội tụ.
Scratch HeadQK ep220	Train lâu hơn giúp Args tăng nhưng Valid JSON và Intent giảm, cho thấy training from scratch không ổn định.
Finetune HeadQK ep220	Cân bằng tốt nhất: JSON và intent cao, Args cao nhất trong nhóm SpikeGPT loss gốc.

6.4 Ablation về hàm mất mát

Bảng 6.3: Kết quả thử copy-aware loss

Cấu hình	Valid JSON	Intent Acc	Args EM	F1
Finetune HeadQK ep220, loss gốc	99.80	94.10	13.94	0.9415
Finetune HeadQK, copy-aware loss	99.94	91.28	16.88	0.9131

Copy-aware loss cải thiện Args Exact Match thêm 2.94 điểm phần trăm. Điều này phù hợp với mục tiêu thiết kế: các token từng xuất hiện trong ngữ cảnh được tăng trọng số nên mô hình chú ý hơn đến khả năng copy. Tuy nhiên, Intent Accuracy giảm từ 94.10% xuống 91.28% và F1-score giảm từ 0.9415 xuống 0.9131. Vì vậy, loss mới không được chọn làm cấu hình chính. Kết quả này vẫn có ý nghĩa vì nó cho thấy nhóm đã thử tối ưu đúng metric yếu nhất, nhưng bài toán có đánh đổi giữa học intent và học exact copying.

Về mặt báo cáo, kết quả này nên được trình bày như một thí nghiệm thất bại có giá trị. Nó chứng minh giả thuyết “tăng trọng số token copy có thể cải thiện Args” là đúng một phần. Nhưng vì tool calling gồm cả chọn tool và sinh arguments, tối ưu quá mạnh một phần mục tiêu có thể làm giảm cân bằng tổng thể. Vì vậy nhóm chọn mô hình Finetune HeadQK epoch 220 làm mô hình chính, còn copy-aware loss là hướng cải tiến cần nghiên cứu thêm.

6.5 Ablation về định dạng ID

Ban đầu nhóm nghi ngờ Args Exact Match thấp do test set sử dụng định dạng ID lệch phân phối train, ví dụ nhiều ID có gạch nối như MAC-59-PR0. Nhóm tạo thêm test set trong đó order_id và product_id được chuẩn hóa về dạng chữ nối số, gần với phân phối train. Kết quả với mô hình Finetune220 đạt 99.88% Valid JSON, 92.86% Intent Accuracy và 13.52% Args Exact Match, không khác biệt đáng kể so với kết quả ban đầu.

Do đó, có thể kết luận định dạng ID không phải nguyên nhân chính. Vấn đề nằm ở năng lực ràng buộc thông tin cụ thể giữa input và output. Nói cách khác, mô hình không chỉ gặp khó với dấu gạch nối, mà gặp khó với việc giữ nguyên một chuỗi token cụ thể qua nhiều bước sinh.

Bảng 6.4: Kiểm tra giả thuyết ID format

Bộ test	Valid JSON	Intent Acc	Args EM
Test OOD ban đầu	99.80	94.10	13.94
Test ID in-distribution	99.88	92.86	13.52

Bảng trên không nhằm chứng minh bộ test nào tốt hơn, mà nhằm kiểm tra một giả thuyết cụ thể: liệu ID có gạch nối nhiều segment có làm model hỏng nặng hay không. Vì kết quả Args EM gần như không tăng khi đưa ID về dạng gần train, giả thuyết này bị bác bỏ. Đây là một điểm mạnh về mặt phương pháp luận vì nhóm không chỉ phỏng đoán nguyên nhân mà có kiểm chứng bằng ablation.

6.6 Phân tích vì sao SpikeGPT yếu ở Args Exact Match

Có bốn nguyên nhân chính.

Thứ nhất, cơ chế hồi quy nén thông tin. SpikeGPT xử lý chuỗi tuần tự qua trạng thái ẩn. Thông tin quá khứ được giữ lại thông qua decay và tích lũy, nhưng không có khả năng truy cập trực tiếp mọi token như self-attention. Với các tham số cần copy chính xác, chỉ cần trạng thái ẩn làm mờ một ký tự hoặc một token, output đã sai.

Thứ hai, biểu diễn xung nhị phân làm giảm độ phân giải. SNN dùng spike 0/1, trong khi Transformer truyền vector số thực liên tục. Biểu diễn nhị phân giúp tiết kiệm năng lượng lý thuyết nhưng có thể làm mất chi tiết vì mô của token, nhất là với chuỗi ID, số lượng và địa chỉ.

Thứ ba, Args Exact Match là metric rất nghiêm ngặt. Nếu model chọn đúng tool và copy đúng phần lớn tham số nhưng sai một ký tự trong address, sai kiểu dữ liệu hoặc thiếu một khóa, toàn bộ mẫu bị tính sai. Vì vậy, khoảng cách giữa Intent Accuracy và Args Exact Match là điều dễ hiểu.

Thứ tư, HeadQK chỉ là attention nhẹ. HeadQK giúp cải thiện khả năng copy nhưng

được thêm ở mức hạn chế để không phá hoàn toàn kiến trúc SpikeGPT. Nó không có sức biểu đạt tương đương nhiều lớp multi-head self-attention của GPT-2.

6.7 Các dạng lỗi thường gặp

Qua quá trình xem log suy luận, nhóm ghi nhận một số dạng lỗi lặp lại.

- **Sai tool nhưng arguments có vẻ hợp lý:** mô hình trích được mã đơn hoặc mã sản phẩm nhưng chọn nhầm giữa `get_order`, `delete_order` và `update_order`.
- **Đúng tool nhưng sai ID:** mô hình sinh đúng schema nhưng bịa hoặc biến đổi một phần `order_id/product_id`.
- **Đúng ID nhưng sai field khác:** trong các tool nhiều tham số như `create_order` hoặc `update_order`, mô hình có thể đúng ID nhưng sai số lượng hoặc địa chỉ.
- **Sai kiểu dữ liệu:** ví dụ số lượng được sinh thành chuỗi thay vì số, làm dictionary không khớp tuyệt đối.
- **Thiếu hoặc thừa khóa:** mô hình sinh thiếu `address` hoặc thêm một argument không có trong ground truth.

Các lỗi này giải thích vì sao Valid JSON và Intent Accuracy cao nhưng Args Exact Match thấp. Một output có thể nhìn có vẻ đúng về cấu trúc, nhưng chỉ cần sai một trường nhỏ là metric Args EM tính sai toàn bộ. Đây cũng là lý do nhóm đề xuất hướng phát triển là tách metric arguments theo từng field để phân tích chi tiết hơn.

Chèn bảng hoặc hình minh họa 3-5 prediction sai tiêu biểu từ predictions.json.

Hình 6.2: Ví dụ các dạng lỗi suy luận thường gặp

6.8 Đánh giá định tính

Về định tính, mô hình SpikeGPT fine-tune cho thấy ba điểm tích cực. Thứ nhất, mô hình thường sinh đúng cặp thẻ `<tool_call>` và `</tool_call>`. Thứ hai, mô hình học được cú pháp JSON tương đối ổn định. Thứ ba, mô hình hiểu được nhiều cách diễn đạt tiếng Việt khác nhau cho cùng một intent, ví dụ “hủy đơn”, “cancel đơn”, “xóa hóa đơn” đều có thể ánh xạ về `delete_order`.

Điểm yếu định tính nằm ở các chuỗi cần giữ nguyên. Khi câu hỏi chứa mã hoặc địa chỉ dài, SpikeGPT có xu hướng sinh một giá trị gần giống nhưng không hoàn toàn trùng. Điều này khác với lỗi ngôn ngữ thông thường: trong hội thoại tự do, một câu gần đúng có thể chấp nhận được; trong tool calling, một mã đơn gần đúng vẫn là sai.

Chèn hình demo thành công: user request -> SpikeGPT sinh đúng JSON tool call.

Hình 6.3: Ví dụ suy luận thành công của SpikeGPT

6.9 So sánh với bài báo gốc

Bài báo SpikeGPT tập trung chứng minh rằng SNN có thể được mở rộng sang mô hình ngôn ngữ sinh, đạt kết quả cạnh tranh trên một số tác vụ NLG/NLU và có tiềm năng tiết kiệm năng lượng. Đề án của nhóm đi theo hướng ứng dụng khác: kiểm tra SpikeGPT trên tool calling tiếng Việt, nơi yêu cầu output có cấu trúc và copy chính xác tham số.

Kết quả của nhóm phù hợp với tinh thần của bài báo ở điểm SpikeGPT có thể học sinh ngôn ngữ và cấu trúc output. Tuy nhiên, đề án cũng chỉ ra một giới hạn thực tế: với bài toán agentic yêu cầu exact copying, SpikeGPT hiện tại chưa cạnh tranh được với Transformer. Điều này không phủ nhận giá trị của SpikeGPT, mà cho thấy cần thêm cải tiến nếu muốn dùng SNN language model cho các tác vụ API calling nghiêm ngặt.

Chương 7

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1 Kết luận

Đồ án đã tìm hiểu, tái hiện và ứng dụng SpikeGPT cho bài toán chatbot agentic trong quản lý thương mại. Nhóm xây dựng bộ dữ liệu tool calling tiếng Việt, fine-tune SpikeGPT-216M, thêm HeadQK, thử sửa dữ liệu và thử copy-aware loss. Kết quả cho thấy SpikeGPT có thể đạt Valid JSON và Intent Accuracy cao, đặc biệt khi fine-tune từ checkpoint pretrained với HeadQK.

Tuy nhiên, điểm yếu lớn nhất là Args Exact Match. So với GPT-2, SpikeGPT thua rõ ở khả năng copy chính xác arguments. Điều này phản ánh giới hạn của kiến trúc hồi quy xung hóa khi phải giữ và tái tạo chính xác các chuỗi token cụ thể. Các thử nghiệm bổ sung cho thấy vấn đề không chỉ đến từ định dạng ID hay lỗi dữ liệu, mà là hạn chế bản chất hơn của mô hình đối với exact copying.

7.2 Hạn chế

- Dataset do nhóm tự sinh nên vẫn có thể còn thiên lệch về template và cách diễn đạt.
- SpikeGPT được giả lập trên GPU, chưa đo được năng lượng thực tế trên phần cứng neuromorphic.
- HeadQK làm tăng độ phức tạp, phần nào giảm ưu thế tiết kiệm của SpikeGPT.
- Args Exact Match là metric nghiêm ngặt nhưng chưa tách riêng lỗi từng tham số như ID, số lượng và địa chỉ.
- Nhóm chưa có đủ tài nguyên để train nhiều seed và nhiều kích thước mô hình lớn hơn.

7.3 Hướng phát triển

- Tách metric arguments theo từng trường để biết mô hình sai chủ yếu ở `order_id`, `product_id`, số lượng hay địa chỉ.
- Thử cơ chế copy/pointer hoặc constrained decoding để đảm bảo tham số được lấy trực

tiếp từ input.

- Kết hợp retrieval hoặc rule-based post-processing cho các trường có yêu cầu exact match.
- Huấn luyện SpikeGPT kích thước lớn hơn hoặc dùng curriculum learning từ tool đơn giản đến tool nhiều tham số.
- Đánh giá trên phần cứng neuromorphic để kiểm chứng ưu thế năng lượng thay vì chỉ dựa trên lý thuyết.

7.4 Bài học rút ra

Bài học quan trọng nhất của đề án là một mô hình có thể sinh JSON hợp lệ và chọn đúng intent nhưng vẫn chưa đủ tốt cho hệ thống agentic thực tế. Trong tool calling, độ chính xác của arguments là yếu tố quyết định. Việc thử nhiều hướng cải tiến, kể cả những hướng không thành công, giúp nhóm hiểu rõ hơn giới hạn của SpikeGPT: mô hình phù hợp với mục tiêu SNN và tiết kiệm năng lượng, nhưng cần thêm cơ chế hỗ trợ copy nếu muốn cạnh tranh với Transformer trong các tác vụ API calling nghiêm ngặt.

Tài liệu tham khảo

- [1] Zhu, R.-J., Zhao, Q., and Eshraghian, J. K. *SpikeGPT: A Generative Pre-trained Language Model with Spiking Neural Networks*. arXiv:2302.13939, 2023.
- [2] Peng, B. et al. *RWKV: Reinventing RNNs for the Transformer Era*. arXiv preprint, 2023.
- [3] Vaswani, A. et al. *Attention Is All You Need*. Advances in Neural Information Processing Systems, 2017.
- [4] Radford, A. et al. *Language Models are Unsupervised Multitask Learners*. OpenAI, 2019.
- [5] Neftci, E. O., Mostafa, H., and Zenke, F. *Surrogate Gradient Learning in Spiking Neural Networks*. IEEE Signal Processing Magazine, 2019.
- [6] Zhu, R.-J. et al. *SpikeGPT official implementation*. GitHub repository: <https://github.com/ridgerchu/SpikeGPT>.